arXiv:2311.02382v2 [cs.DC] 8 Nov 2023

ULTRA-LONG SEQUENCE DISTRIBUTED TRANSFORMER

Xiao Wang [1] **Isaac Lyngaas** [1] **Aristeidis Tsaris** [1] **Peng Chen** [2] **Sajal Dash** [1] **Mayanka Chandra Shekar** [1]
Tao Luo [3] **Hong-Jun Yoon** [1] **Mohamed Wahib** [4] **John Gounley** [1]

ABSTRACT

Transformer models trained on long sequences often achieve higher accuracy than short sequences. Unfortunately, conventional transformers struggle with long sequence training due to the overwhelming computation and memory requirements. Existing methods for long sequence training offer limited speedup and memory reduction, and may compromise accuracy. This paper presents a novel and efficient distributed training method, the Long Short-Sequence Transformer (LSS Transformer), for training transformer with long sequences. It distributes a long sequence into segments among GPUs, with each GPU computing a partial self-attention for its segment. Then, it uses a fused communication and a novel double gradient averaging technique to avoid the need to aggregate partial self-attention and minimize communication overhead. We evaluated the performance between LSS Transformer and the state-of-the-art Nvidia sequence parallelism on a Wikipedia *enwik8* dataset. Results show that our proposed method lead to 5.6x faster and 10.2x more memory-efficient implementation compared to state-of-the-art sequence parallelism on 144 Nvidia V100 GPUs. Moreover, our algorithm scales to an extreme sequence length of 50,112 at 3,456 GPUs, achieving 161% super-linear parallel efficiency and a throughput of 32 petaflops.

1 INTRODUCTION

The transformer is a powerful neural network architecture widely used in natural language and image processing (Vaswani et al., 2017). Its versatility is evidenced by its wide range of applications, including machine translation (Wang et al., 2019), chatbots (Caldarini et al., 2022), speech recognition (Dong et al., 2018), image captioning (Yu et al., 2019), image segmentation (Valanarasu et al., 2021; Strudel et al., 2021), and classification (Chen et al., 2021b). The transformer achieves its impressive performance by recognizing that different input sequence tokens have varying levels of importance to the final output prediction. The transformer captures the relationship between each pair of input tokens using a process called *"self-attention"*. This allows the transformer to generate highly accurate outputs by focusing on the most relevant tokens in an input sequence while also paying attention to the overall context. This approach has proven to be highly effective and makes transformer a leading technology in artificial intelligence.

With long sequence training, transformer attends to many more input tokens than a transformer trained with short sequences. Therefore, long sequence training often captures more contextual information and leads to markedly higher prediction accuracy for many tasks with long-range dependencies, such as DNA sequence analysis (Zaheer et al., 2020), long document summary (Beltagy et al., 2020) and image segmentation (Strudel et al., 2021). Unfortunately, transformer's memory footprint increases quadratically and computations increase cubically with longer sequence lengths (Beltagy et al., 2020; Dao et al., 2022). Therefore, the transformer's sequence length is typically truncated to no more than a couple thousand tokens due to runtime and memory constraints, despite longer sequences leading to higher accuracy.

To address this issue, there are three research approaches: hierarchical training, attention approximation, and distributed sequence parallelism. Hierarchical training involves training multiple transformers at different levels of abstraction (Si & Roberts, 2021; Chen et al., 2022; 2021b; Yu et al., 2023). The transformer at the lowest abstraction level trains on the shortest sequence segments. Then, the transformer at the next higher level uses the previous level outputs as additional input to train on longer segments. The process then repeats until reaching the highest abstraction level. However, training multiple transformers at different abstraction levels significantly increases training time and memory footprint. In addition, hyper-parameter tuning is required for the optimal model architecture at each abstraction level.

The approximation approach, in contrast, aims to reduce the computations and memory usage by approximating the self-

1, Oak Ridge National Laboratory, US. 2, National Institute of Advanced Industrial Science and Technology, Japan. 3, Agency for Science, Technology and Research, Singapore. Corresponding email: wangx2@ornl.gov

Table 1. A summary for different long sequence training methods. *Serial* methods may need to be used on top of parallelism schemes. l_x= sequence length; N= number of GPU workers; H= number of hierarchical levels; Z= number of non-zeros.

Approach	Method	Accuracy Loss	Serial/ Distributed	Memory (per worker)	Compute (per worker)	Memory Total	Compute Total	Comm. freq. per layer	Practical bottleneck
Hierarchical Training	(Chen et al., 2022; 2021b) (Si & Roberts, 2021; Yu et al., 2023)	No	Serial	$O(l_x^2 H)$	$O(l_x^3 H)$	N/A	N/A	N/A	Train multiple models
Attention Approximation	(Kitaev et al., 2020; Roy et al., 2021) (Child et al., 2019; Beltagy et al., 2020)	Yes	Serial	$O(Z)$	$O(Z^3)$	N/A	N/A	N/A	Low sparsity
Sequence Parallel	Straightforward Seq. Parallel (Li et al., 2021; 2023)	No	Distributed (partial aggr.)	$O(l_x^2/N)$	$O(l_x^3/N)$	$O(l_x^2)$	$O(l_x^3)$	$\approx N^2$	Frequent communication
	Nvidia Seq. Parallel (Korthikanti et al., 2022)		Distributed (serial attention)	$O(l_x^2)$	$O(l_x^3)$	$O(l_x^2)$	$O(l_x^3)$	8	Attention not distributed
	LSS Transformer (ours)		Distributed (fully)	$O(l_x^2/N)$	$O(l_x^3/N)$	$O(l_x^2)$	$O(l_x^3)$	2	-

attention operations through either sparse sampling (Child et al., 2019; Kitaev et al., 2020; Roy et al., 2021; Beltagy et al., 2020; Zaheer et al., 2020), low-rank approximation (Choromanski et al., 2021; Katharopoulos et al., 2020), infrequent self-attention updates (Ying et al., 2021; Rabe & Staats, 2022), or their combinations (Chen et al., 2021a). These approximation methods can significantly reduce memory footprint and computations, and some can even reduce the long sequence problem's quadratic complexity to linear complexity. However, approximation is a lossy information compression technique that discards partial information for the self-attention. Thereby, excessive approximation may lower accuracy especially for sequences with long-range dependency. Previous experiments demonstrate significant accuracy degradation when approximation compression ratio exceeds 70% (Shi et al., 2021).

The distributed sequence parallelism approach aims to address the long sequence problem by distributing long sequences into contiguous sequence segments among GPUs (Li et al., 2021; 2023; Korthikanti et al., 2022; Jacobs et al., 2023). The largest hurdle in distributed sequence parallelism lies in handling the self-attention, which is both the most compute-intensive but also the most communicate-intensive step. Since each GPU's sequence segment has a correlation with every other GPU's segment, a straightforward distributed self-attention method, such as (Li et al., 2021), Deep-Speed Ulysses (Jacobs et al., 2023) and the most recent LightSeq (Li et al., 2023), requires each GPU to communicate with every other GPU multiple times to calculate a partial self-attention score for its assigned segment, before aggregating them into the final self-attention output. Consequently, communication frequency tends to increase significantly in a quadratic growth rate with more sequence parallel GPUs, substantially impeding scalability.

Conversely, an alternative sequence parallel method from Nvidia's Megatron-LM framework completely avoids self-attention dependency by performing sequential updates on the core self-attention and feed-forward operations along the sequence dimension, while parallelizing less compute-intensive but independent tasks, such as layer normalization and dropouts (Korthikanti et al., 2022). Although this method requires only 8 communications per attention layer, it misses the opportunity to parallelize the compute intensive self-attention. Consequently, it leads to a modest 29% speedup compared to a sequential baseline with full backward recomputation (Korthikanti et al., 2022).

A summary of each approach's core idea and limitations is provided in Table 1. To address their limitations, this paper introduces the "Distributed Long Short-Sequence Transformer" (LSS Transformer). In contrast to existing methods, the LSS Transformer (1) utilizes a single transformer for training; (2) attains remarkable speedup and memory reduction; (3) has no approximation, thereby no accuracy loss; and (4) maintains a minimal communication overhead, requiring only 2 communications per attention layer. It distributes a long sequence into short segments among GPUs, with each GPU computing a partial self-attention score for its segment in the context of the entire sequence. Then it uses a fused communication and a double gradient averaging technique to avoid the need to aggregate partial self-attention score, minimize communication overhead while preserving data dependency.

This paper contributes in the following ways: (1) Introducing a general and innovative end-to-end sequence framework for long sequence training. Operating at the attention layer level, it remains agnostic to model sizes and variations (encoder-only, decoder-only, etc.), making it universally applicable without modifications. (2) Presenting a low communication overhead algorithm that uses fused communication and gradient averaging to lower communication frequency. (3) Presenting an integrative sequence and data parallelism algorithm that minimizes inter-GPU communications within local communicative groups. Our assessment on the Wikipedia *enwik8* dataset demonstrates the superiority of the LSS Transformer over state-of-the-art Nvidia sequence parallelism (Korthikanti et al., 2022), achieving 6x faster training and 10x enhanced memory efficiency on 144 Nvidia V100 GPUs. Moreover, our algorithm scales remarkably to an extensive sequence length of 50,112 using 3,456 GPUs, delivering 161% super-linear parallel efficiency and a computation throughput of 32 petaflops.

Table 2. Three distinct challenges for training transformer and their orthogonal levels of parallelism. N_d = data size; N_m = model size; l_x = sequence length; B = batch size;

Challenges	Large Dataset	Large Model Size	Long Sequences
Computational complexity	$O(N_d)$	$O(N_m)$	$O(l_x^3)$
Memory complexity	$O(B)$	$O(N_m)$	$O(l_x^2)$
Parallelism	Data Parallel	Model Parallel	Sequence Parallel
Source of parallelism	Distributed input data batches	Distributed model parameters	Distributed sequence segments
Distributed memory	No	Yes	Yes

2 Background and Motivation

2.1 Orthogonal Levels of Parallelism

Training a transformer model has three distinct computation challenges: (1) large training dataset, (2) large model size and (3) long sequences. Table 2 offers a concise overview of these challenges and their characteristics. As discussed earlier, increasing sequence length leads to a cubic growth rate for computations and a quadratic growth rate for memory use. In contrast, scaling model parameters or data batches presents linear computational and memory complexities.

Each challenge relies on a unique parallelism. These parallelism levels are mostly orthogonal, address different challenges and cannot replace each other. Data parallelism accelerates training on large datasets by distributing training batches. However, it does not distribute memory and each GPU has a copy of the entire model. Model parallelism achieves speedup and distributed memory for large model problem by distributing model parameters and their gradients. Sequence parallelism, in contrast, accelerates computations and distributes memory for long sequences by distributing sequence segments. This paper parallelizes in the sequence dimension to scale the sequence length by distributing memory and compute cost, while also carefully assuring that the sequence parallelism does not interfere with or hinders other forms of parallelization.

2.2 Transformer Architecture

Figs. 1(i)-(iii) illustrate a standard transformer architecture for both the decoder only (i.e., GPT) and the encoder only (BERT) models. A token embedding in Fig. 1(i) converts a tokenized image or text input sequence x into an input vector. The input vector is then enhanced with embedded positional information.

Next, the input vector x of size $l_x \times E_m$, with l_x representing the sequence length and E_m being the embedding size, undergoes layer normalization and random dropouts before being processed by the self-attention. The detailed steps for self-attention are illustrated in Fig. 1(ii) with the goal of producing a contextual embedding for each input token in relation to the whole sequence. In the self-attention unit, the input vector x is linearly transformed into query (Q), key (K), and value (V) vectors, with all three vectors having the same size as input x. Then self-attention computes the following equation:

$$E = \text{softmax}\left(QK^T/\sqrt{d_k}\right)V = A_w V \ . \qquad (1)$$

The self-attention score, denoted as A_w, has size $l_x \times l_x$ and quantifies the correlation between each pair of tokens. This correlation is calculated by taking the dot product between Q and the transposed K. To stabilize gradients during training, the self-attention score A_w is further scaled by a constant $\sqrt{d_k}$, and is then normalized by a dropout and a SoftMax activation. The final self-attention output E is obtained by weighted averaging between value vector V and A_w. The output embedding E is then linearly transformed before exiting the self-attention unit.

The contextual embedding output from self-attention is further processed by a feed-forward unit, with detailed steps illustrated in Fig. 1(iii), residual connections, and layer normalization to enhance training stability and prediction accuracy. These components, along with dropouts, form a single attention layer, outlined by a gray box in Fig. 1(i). Then the attention layers repeat itself by L times, where L is the number of attention layers. After undergoing final layer normalization and linear transformation, the output y is compared to the target $\tilde{y}$ using cross-entropy loss, and all parameters are updated.

It's crucial to note that self-attention dominates both computation and memory requirements for training. With a size of $l_x \times l_x$, the self-attention score, A_w, has quadratic expansion in its size with increased sequence length. This leads to transformer's cubic computational and quadratic memory complexities with respect to sequence length.

2.3 State-of-the-Art Sequence Parallelism

To parallelize self-attention, the straightforward distributed self-attention method, such as (Li et al., 2021; 2023), partitions both the input vector x and its linearly transformed vectors Q, K and V into distributed segments among GPUs. For example in a scenario with 3 GPUs, GPU 1 receives the first segments of these vectors, namely x_1, Q_1, K_1, and V_1, and GPUs 2 and 3 receive the second and the third

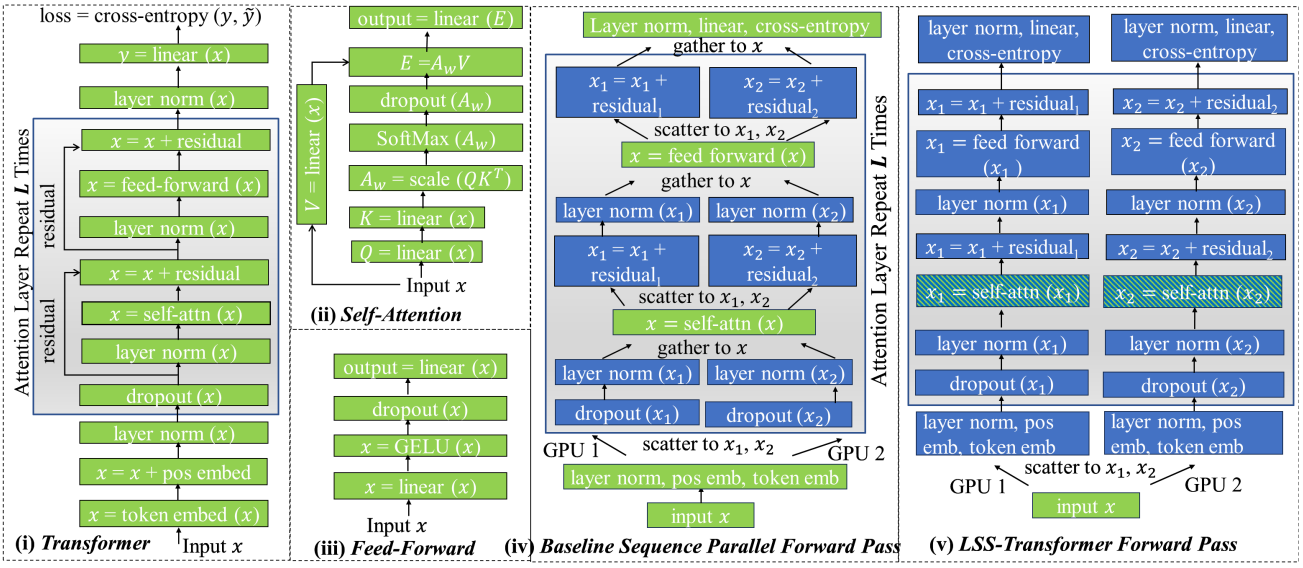


Figure 1. (i) A generic transformer with L attention layers outlined by a gray box. (ii) and (iii) the self-attention and the feed-forward units, respectively. (iv) Forward pass for baseline sequence parallelism with 2 sequence distributed GPUs. Blue indicate distributed steps and green for sequential steps. (v) The 2 GPU example for the LSS Transformer's forward pass.

segments. To compute Eqn. (1), each GPU must receive every other GPU's segment to compute partial self attention scores. For example, GPU 1 needs to receive K_2 and K_3 from GPUs 2 and 3 before GPU 1 can compute partial self-attention scores $Q_1K_2^T$ and $Q_1K_3^T$. Then, the partial self-attention scores are aggregated across GPUs into the complete self-attention score, which is then used in dot products with the distributed value vector V. Since each GPU must communicate with every other GPU multiple times, the communication frequency for the straightforward distributed self-attention method increases quadratically with more GPUs, significantly limiting its scalability.

To address this limitation, Nvidia's method (Korthikanti et al., 2022), referred to as "baseline sequence parallelism" in the rest of the paper, computes self-attention and feed-forward sequentially to avoid the quadratically increased cross-GPU communications (Korthikanti et al., 2022). However, it independently parallelizes layer normalization and dropouts in the sequence dimension, as they lack such inter-token dependencies (Korthikanti et al., 2022). Note that the term "sequential computations" pertains to single GPU computations, while "parallel computations" refer to those performed across GPUs in the sequence dimension. Although self-attention and feed-forward are computed within a single GPU, their computations are still vectorized through parallel threads within the GPU.

Fig. 1(iv) summarizes the baseline sequence parallelism using an example of 2 GPUs. During a forward pass, positional and token embeddings are computed sequentially and the output is scattered into contiguous segments among the GPUs along the sequence dimension. Then in the attention layers, the feed-forward and the self-attention are sequentially updated by a single GPU, and are represented by green rectangles in the figure. All other steps are independently updated with sequence parallelism, represented by blue rectangles. Gather and scatter communications are used before and after self-attention and feed-forward to ensure sequential updates for them and independent parallel updates for all other steps. Finally, the results from GPUs are gathered for a final layer normalization, linear transform, and cross-entropy loss evaluation. In the backward pass, all steps are the same as the forward pass, except the gather communications in the forward pass is replaced by reduce-scatter for gradient synchronization among GPUs and scatter communications in the forward pass are replaced with gather operations in the backward pass.

Despite that the baseline sequence parallelism avoid the frequent GPU communications in self-attention as in the straightforward distributed method, it has two main limitations. First, the most compute-intensive self-attention and feed-forward steps are sequential. Secondly, the communication overhead is still significant with 8 global communications per attention layer (4 in forward pass and 4 in backward pass). As a result, the baseline sequence parallelism achieves only 3% speedup on a 22-billion-parameter model compared to a baseline without backward pass recomputation, and up to 29% speedup compared to a baseline with backward recomputation (Korthikanti et al., 2022).

3 The Distributed Long Short-Sequence Transformer

To achieve excellent scalability, the LSS Transformer must distribute self-attention, but also overcome the quadratically increased communication frequency. The LSS Transformer performs sequence parallelism based on these principles:

Principle 1: Independent Computations with Distributed Memory. Except for self-attention, all other computations like layer normalization, residual connection, dropout, feed-forward, linear transform, positional, and token embedding can be independently computed and distributed among GPUs without dependencies in the sequence dimension. Memory storage for these steps is also distributed in the sequence dimension in the same manner.

The feed-forward operation can be independently computed by row-distributing its linear transform multiplication along the sequence length dimension, enabling independent GPU computations. Additionally, GeLu activation and dropout operate independently on each element of their input. For instance, for a sequence input x of dimension $l_x \times E_m$, where l_x is sequence length and E_m is embedding size, the first linear transform step in the feed-forward unit, $linear(x)$ in Fig. 1(iii), multiplies x with linear transform model parameters of size $E_m \times D_{inner}$, where D_{inner} is the dimension of the feed-forward hidden layer. This matrix multiplication can be row-distributed among GPUs without communication when input x is distributed into sequence segments x_i of size $\frac{l_x}{N} \times E_m$, where N is the number of sequence parallel GPUs. After independent element-wise operations on GeLu and dropout, the sequence distributed output is used as distributed input for the second linear transform multiplication in the feed forward unit, again row-distributed without communication.

Fig. 1(v) depicts the LSS Transformer's forward pass, showcasing a demonstration with 2 sequence parallel GPUs. Note that while the figure exemplifies a specific scenario, the LSS Transformer's operate at the attention layer level, and is universally adaptable to various model sizes and types without modification. In this figure, input sequence x is scattered to x_1 and x_2 in the sequence dimension and each GPU receives a segment. Principle 1 enables all subsequent operations to be sequence distributed among GPUs and computed independently. In addition, each operations' inputs, intermediate outputs and their associated gradients are also stored in distributed memory across sequence-parallel GPUs, enabling excellent memory footprint reduction. We use blue rectangles in the figure to represent these independent computation steps. The self-attention is marked by a shaded blue rectangle to indicate that self-attention is distributed yet requires inter-GPU communications.

Principle 2: Sequence Distributed Positional Embedding. The positional embedding parameters are a lookup table and each row of the table represents a token's spatial position within the sequence. The number of rows of the lookup table corresponds to the sequence length. Since each sequence-distributed GPU receives contiguous sequence segments, the GPU performs lookup operations only on the corresponding contiguous rows of the lookup table. This allows for row distribution of the positional embeddings among the GPUs without dependency.

Principle 3: Distributed Self-Attention with Fused Communications. To parallelize self-attention, we use the following math property to preserve data dependency while minimizing communication overhead. By distributing the query vector, Q, in the sequence dimension among GPUs, we compute the self-attention output, E, as the following concatenation in the sequence dimension, and in parallel:

$$E = \begin{bmatrix} \text{softmax}\left(Q_1 K^T \,/\, \sqrt{d_k}\right) V \\ \text{softmax}\left(Q_2 K^T \,/\, \sqrt{d_k}\right) V \\ \text{softmax}\left(Q_3 K^T \,/\, \sqrt{d_k}\right) V \\ \cdots \end{bmatrix}, \tag{2}$$

where Q_i is an $\frac{l_x}{N} \times E_m$ distributed query vector segment received by the ith GPU, where N is the number of GPUs. V and K are $l_x \times E_m$ collected value and key vectors without distribution, having the same copy across all GPUs. $\text{softmax}(Q_i K^T / \sqrt{d_k})$ represents the ith GPU's partial self-attention score for its assigned segment, but in the context of the whole sequence.

In summary, the key idea is to distribute the query vector in the sequence dimension among GPUs, while keeping the value and key vectors collected. Then LSS Transformer computes an individual self-attention output, $\text{softmax}(Q_i K^T / \sqrt{d_k}) V$, for each GPU. Eqn. (2) shows that concatenating GPUs' individual self-attention outputs is ***numerically the same*** as directly computing self-attention E sequentially, since the concatenation is a simple row gathering for E. Therefore, our proposed distributed self-attention method is an exact method without approximation, thereby no accuracy loss. Compared to the straightforward sequence parallelism with quadratically increased communication frequency, a significant advantage of Eqn. (2) is that it enables distributed computing while requiring only 6 communications per self-attention layer. The forward pass requires 2 communications from gathering value and key vectors and 1 from self-attention concatenation. The backward pass requires another 3 communications.

To further reduce communication overhead, we use a fused communications technique to reduce the communication frequency from 6 communications per layer to 4. Fig. 2(i) demonstrates the operations in the forward pass without the fused communication. An example sequence segment, x_i

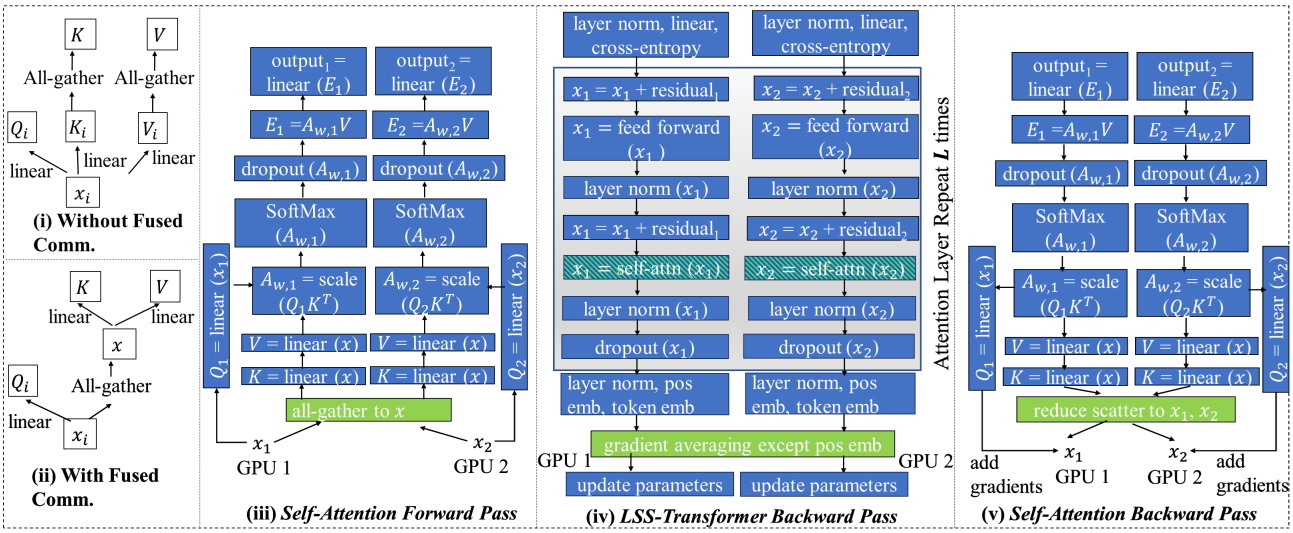


Figure 2. (i) and (ii) show the difference without and with fused communications. (iii) shows distributed self-attention's forward pass with fused communications. Note that the distributed self-attention outputs are not concatenated. (iv) LSS Transformer's Backward pass. Model parameters, except the positional embedding, are synchronized through gradient averaging. (v) The distributed self-attention's backward pass with reduce-scatter.

is linearly transformed into query, key and value segments. Then, two all-gather communications are independently operated on the key and value segments into the collected K and V. Fig. 2(ii) shows the fused communication operation in the forward pass, requiring only a single all-gather communication. x_i is linearly transformed into query segment Q_i. Meanwhile, x_i is gathered into a temporary collected sequence x, before x is linearly transformed into the collected key and value vectors. The same technique is also applied to backward pass, reducing the total number of communications from 6 to 4 per attention layer.

Principle 4: Gradient Averaging Technique to Synchronize GPUs and Avoid Concatenation. There are two issues from Principles 1 and 3. First, since sequence parallel GPU trains on the same model parameters but using different input sequence segments, the gradients for the model parameters are different for each GPU. The second issue is that the self-attention communication frequency needs to be further reduced to achieve even better scalability and parallel efficiency.

To address both issues, we use a gradient averaging technique to synchronize model parameters and avoid the concatenation for the GPUs' individual self-attention outputs. Therefore, communication frequency is reduced from 4 to 2 per attention layer. Figs. 2(iii)-(v) use a 2 GPU example to demonstrate how this gradient averaging technique is applied. In the forward pass for the self-attention in Fig. 2(iii), a distributed query Q_i is computed from the input sequence segment x_i. Meanwhile, the self-attention input segments are gathered among GPUs before computing collected K and V vectors using a single all-gather fused communication, as explained before in Principle 3. Subsequent computations and memory storage are all distributed and independently updated in the sequence dimension, producing individual self-attention output for each GPU.

The individual self-attention outputs, however, are not concatenated across GPUs in Fig. 2(iii). Instead, the LSS Transformer allows each GPU to use its assigned sequence segment and individual self-attention output to compute a partial cross-entropy loss and gradients in the backward pass in Figs. 2(iv) and (v). Note that the backward pass in Fig. 2(v) uses reduce-scatter as the backward operation for the all-gather in the forward pass. Finally, the averaged gradients are computed and used for synchronized model parameter updates before training on the next data batch. One important technical detail to mention is that the averaged gradients are not computed for the positional embeddings, which are distributed parameters across GPUs and should not be synchronized.

To understand why this gradient averaging technique can avoid self-attention concatenation and synchronize model parameters at the same time, let us assume that the predicted sequence output from transformer is y and its true label is $\tilde{y}$. The cross-entropy loss for the whole sequence, denoted as $L(y, \tilde{y})$, equals the average of individual token's loss: $L(y, \tilde{y}) = \frac{1}{l_x}\sum_{i=1}^{l_x} L(y_i, \tilde{y}_i)$, where l_x is sequence length. According to the gradient summation rule, the gradient of $L(y, \tilde{y})$ with respect to model parameters, denoted as $\nabla L(y, \tilde{y})$, equals the averaged gradient of each token's loss: $\nabla L(y, \tilde{y}) = \frac{1}{l_x}\sum_{i=1}^{l_x} \nabla L(y_i, \tilde{y}_i)$. Therefore, there is no

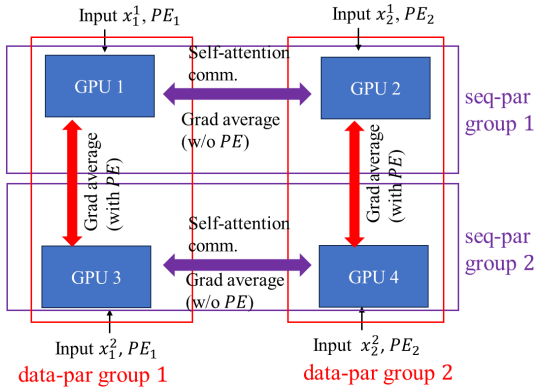


Figure 3. Integrated sequence and data parallelisms with double gradient averaging. The horizontal direction gradient averaging synchronizes parameters without positional embeddings, and the vertical direction gradient averaging includes positional embeddings.

need to concatenate individual self-attention outputs to compute the loss and gradients for the whole sequence. Instead, each GPU uses its scattered individual self-attention output to compute a partial loss and gradient for each sequence segment, before averaging each segment's gradients for a synchronized model parameters update.

By avoiding the expensive concatenation operations in each attention layer, LSS Transformer lowers its communication frequency to only twice per attention layer (one all-gather in forward pass and one reduce-scatter in backward pass) since the gradient averaging occurs only once per data batch. This results in much better scalability and reduced communications compared to other sequence parallel methods.

4 Integrated Sequence & Data Parallelism

The LSS Transformer's sequence parallelism has three limitations. First, it still requires 2 global inter-GPU communications per attention layer, which degrades parallel efficiency at many GPUs. Second, while sequence parallelism tackles the long sequence issue, it does not address computation challenge for training large dataset. Three, sequence parallelism is only one source of parallelism. To scale to a large supercomputer for training, the LSS Transformer needs more sources of parallelism to achieve better scalability. To address these issues, this section introduces a method to integrate the LSS Transformer's sequence parallelism with data parallelism. With the integration, the parallel algorithm can (1) achieve better scalability; (2) simultaneously tackle long sequence and large dataset challenges; and (3) constrain the self-attention communications among local communicative groups for reduced overhead.

Despite that sequence and data parallelisms are mostly orthogonal, one technical challenge to overcome is that both parallelisms require model parameter synchronization, but among GPUs in different communicative groups and communicate in different ways. Sequence parallelism requires model parameter synchronization among sequence parallel GPUs, but excludes positional embedding parameters from synchronization given that positional embeddings are distributed in sequence dimension. Data parallelism requires model parameter synchronization among data parallel GPUs, but must include positional embeddings given that data parallel GPUs have the same copy of the positional embedding parameters, but train them with different data batches.

To address this issue, we use an innovative double gradient averaging technique to avoid synchronization conflicts for positional embeddings. Fig. 3. illustrates an example of how the integrated sequence and data parallelism uses double gradient averaging. In this example, GPUs 1 and 2 process a sequence x^1 together using sequence parallelism, with the first segment x_1^1 assigned to GPU 1 and the second segment x_2^1 assigned to GPU 2. The positional embedding parameters are distributed in the same way with the first half PE_1 assigned to GPU 1 and the second half PE_2 assigned to GPU 2. Similarly, GPUs 3 and 4 handle a difference sequence x^2 using sequence parallelism.

All GPUs that process the same sequence form a sequence parallel group and each group is shown as a horizontal purple box in Fig. 3. Each sequence parallel group's cross-GPU communications, shown as a horizontal purple arrow, are local and confined among GPUs in the same sequence parallel group. These communications involve a fused all-gather and a reduce-scatter in each attention layer for computing distributed self-attention. In addition, a gradient averaging is required once per data batch for model parameters synchronization and avoiding self-attention output concatenation, as discussed before in Sec. 3. The positional embedding parameters, however, are excluded from gradient averaging, as they are distributed across sequence parallel GPUs, and should not be synchronized.

Meanwhile, GPUs 1 and 3 are in a data parallel group, shown as a vertical red box in the figure, and GPUs 2 and 4 are in another data parallel group. GPUs in the same data parallel group process sequence segments from different data batches, but these sequence segments share the same spatial position within their sequences, thereby sharing the same positional embedding. The only needed inter-GPU communications in the same data parallel group, shown as vertical red arrows in the figure, are gradient averaging to synchronize parameters trained with different batches. Similar to sequence parallel groups, the communication for the data parallel groups is also localized and confined within each group. The gradient averaging for data parallelism,

Table 3. The LSS Transformer and Nvidia baseline sequence parallelism's weak scaling experiment for the small model experiment, using only 1 data parallel group.

(a) LSS Transformer, data parallel group = 1

Nodes	1	6	18	54	144
GPUs	6	36	108	324	864
Sequence Groups	6	36	108	324	864
Sequence Length	348	2088	6264	18792	50112
Per GPU Mem. (GB)	0.54	1.01	2.05	5.94	13.58
FLOPS (x10^{12} flop/s)	8	189	881	3000	8245
Self-Attn Comp Incr.	1	6	18	54	144
Parallel Efficiency	100%	165%	174%	173%	151%

(b) Baseline, data parallel group = 1

Nodes	1	6	18
GPUs	6	36	108
Sequence Groups	6	36	108
Sequence Length	348	2088	6264
Per GPU Mem. (GB)	0.94	10.29	OOM
FLOPS (x10^{12} flop/s)	5	32	OOM
Self-Attn Comp Incr.	1	6	OOM
Parallel Efficiency	100%	42%	OOM

Table 4. The LSS Transformer and Nvidia baseline weak scaling for the small model experiment, using 4 data parallel groups.

(a) LSS Transformer, data parallel groups = 4

Nodes	4	24	72	216	576
GPUs	24	144	432	1296	3456
Sequence Groups	6	36	108	324	864
Sequence Length	348	2088	6264	18792	50112
Per GPU Mem. (GB)	0.54	1.01	2.11	5.94	13.58
FLOPS (x10^{12} flop/s)	28	703	3319	10987	32784
Self-Attn Comp Incr.	1	6	18	54	144
Parallel Efficiency	100%	167%	173%	159%	161%

(b) Baseline, data parallel groups = 4

Nodes	4	24	72
GPUs	24	144	432
Sequence Groups	6	36	108
Sequence Length	348	2088	6264
Per GPU Memory (GB)	0.94	10.29	OOM
FLOPS (x10^{12} flop/s)	18	126	OOM
Self-Attn Comp Incr.	1	6	OOM
Parallel Efficiency	100%	42%	OOM

however, must include positional embeddings for synchronization, given that the training segments in the same data parallel group share the same positional embedding.

5 RESULTS

5.1 Experiment Setup

Dataset: *enwik8* is a popular 100-million-byte Wikipedia XML character training dataset (Hutter et al., 2006; Beltagy et al., 2020). Initially used as the testing dataset for the Hutter Prize, the dataset can be downloaded at (Mahoney, 2006) for public benchmark evaluation with a score board available at (with Codes, 2022).

Computing platform: Experiments were conducted on Oak Ridge National Lab's Summit supercomputer, which has 6 NVIDIA V100 GPUs and 2 POWER9 CPUs for each node. Nodes are connected via Mellanox EDR 100G InfiniBand Non-blocking Fat Tree network. Each POWER9 CPU in the node is densely-connected to 3 GPUs with Nvidia NVlinks, where each link has 100 GB/s bidirectional bandwidth, and the two CPUs for each node are connected via an X bus with 64 GB/s bidirectional bandwidth. Each CPU has 22 cores (4 hardware threads for each) and 256 GB DRAM memory. Each GPU has 80 streaming multiprocessors and 16 GB memory. There are additional 54 "high memory" nodes, which has 32 GB of memory per GPU.

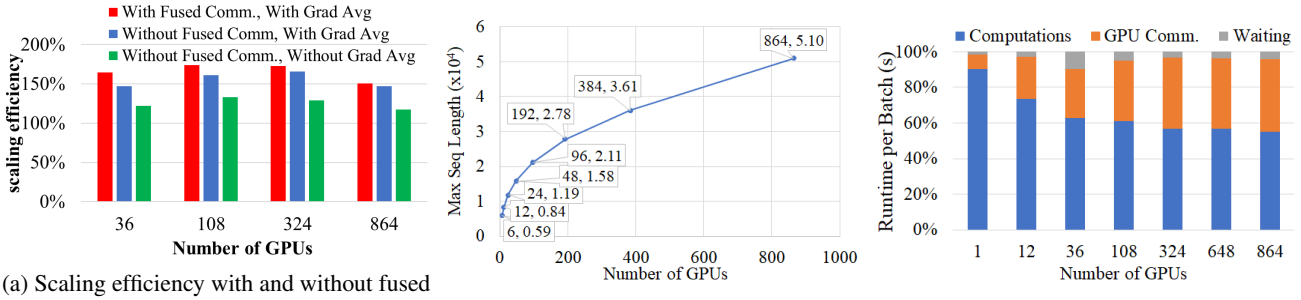
Software: The LSS Transformer is developed in PyTorch and will be made publicly available in the next revision.

5.2 Small Model Experiment

Model: All experiments in the small model experiment subsection trains a decoder-only transformer (GPT) with a 512 embedding size, 6 attention layers and 8 multi-heads for a total of 20 million parameters. The input data batch size is 4. We choose this model size for two reasons. First, this is the standard model for the *enwik8* benchmark evaluation with an excellent bits-per-character accuracy score at 1.0 (Beltagy et al., 2020; Al-Rfou et al., 2019; Sukhbaatar et al., 2019). Second, choosing a small model size allows us to maximize memory usage and evaluate performance for scaling long sequences, instead of maximizing memory for storing parameters for a large model.

Sequence Parallelism Weak Scaling: Tables 3(a) and 3(b) are the weak scaling performance comparison between the LSS Transformer and Nvidia baseline sequence parallelism. We increased both the sequence lengths and the number of sequence parallel GPUs at the same rate, while keeping the number of data parallel group to be 1.

The first two rows of the tables indicate the number of nodes and GPUs, with 6 GPUs per node. The third row represents the number of sequence parallel groups, which is equal to the number of GPUs in this case as the number of



(a) Scaling efficiency with and without fused communication and gradient averaging

(b) Maximum sequence length

(c) Runtimes breakdown for Table 3(a)

Figure 4. (a) Scaling efficiency with and without fused communication and gradient averaging. (b) maximum sequence length at different numbers of sequence parallel GPUs. (c) the runtime breakdown for the LSS Transformer's small model experiment weak scaling with only 1 data parallel group.

Table 5. Performance comparison between the two algorithms on the small model, using only 1 data parallel group. Sequence lengths increase sub-linearly with a growth rate proportional to the square root of the number of GPUs.

(a) LSS Transformer, data parallel group = 1

Nodes	1	6	18	54	144
GPUs	6	36	108	324	864
Sequence Groups	6	36	108	324	864
Sequence Length	366	900	1512	2592	4320
Per GPU Mem. (GB)	0.53	0.62	0.74	0.89	1.15
FLOPS (x10^{12} flop/s)	8	83	266	673	1280
Self-Attn Comp Incr.	1	2	4	6	10
Parallel Efficiency	100%	120%	114%	96%	72%

(b) Baseline, data parallel group = 1

Nodes	1	6	18
GPUs	6	36	108
Sequence Groups	6	36	108
Sequence Length	366	900	1512
Per GPU Mem. (GB)	0.89	2.55	5.86
FLOPS (x10^{12} flop/s)	5	28	71
Self-Attn Comp Incr.	1	2	4
Parallel Efficiency	100%	56%	36%

data parallel group is 1. The fourth row shows the sequence lengths, increasing proportionally with the number of GPUs.

The fifth row displays the average per-GPU peak memory footprint in Gigabytes (GBs). At 6 nodes, the LSS Transformer's per-GPU memory footprint is 1.01 GB, and it scales to 144 nodes with 13.58 GB per GPU. Since Transformer has a quadratic memory complexity of $O(l_x^2/N)$, where l_x is sequence length and N is the number of GPUs, increasing sequence length l_x and number of GPUs N at the same rate will still lead to a linear increase of memory. This explains why LSS Transformer has a small memory footprint at 1 node but increases to much larger memory footprint at more nodes. In comparison, the baseline sequence parallelism has a per-GPU memory footprint of 10.29 GB at 6 nodes, over 10 times larger than that of the LSS Transformer at the same nodes. The baseline sequence parallelism cannot scale beyond 6 nodes due to memory constraint, resulting in "OOM" (out of memory).

The sixth row represents the number of single-precision floating point operations (FLOP) across all GPUs in a second. The LSS Transformer achieves a computation throughput of 8 petaflops at 144 nodes with sequence length of 50,112. In comparison, the baseline sequence parallelism is 5.9 times slower at 6 nodes, achieving a throughput of 32 teraflops, and cannot scale further due to memory constraint.

The seventh row shows the recorded per-GPU computations increase for the self-attention unit relative to the per-GPU computations at 1 node. Since transformer has a cubic computation complexity, distributing computations across GPUs will still lead to computation increase for weak scaling.

The eighth row represents parallel efficiencies, which is the ratio between the actual speedup and the ideal speedup. The LSS Transformer maintains a 151% super-linear parallel efficiency at 144 nodes, while the baseline sequence parallelism's efficiency drops to 42% at only 6 nodes.

Integrated Sequence & Data Parallelism Weak Scaling: To understand how the integration of data and sequence parallelisms accelerates training speed and reduces communication overhead, Table 4(a) repeats the same experiment as Table 3(a), but with 4 data parallel groups. This means that the total number of GPUs quadruples accordingly, but the numbers of sequence parallel groups remain the same as before. By comparing the results between the 4 data parallel groups and the single data parallel group in Tables 3 and 4, we observe that the FLOPs throughput increases by almost 4 times from 1 to 4 data parallel groups with 4 times more GPUs, achieving 32 petaflops at 3456 GPUs. This result indicates that the proposed local communication scheme en-

ables the integrated sequence and data parallelism with little additional communication overhead and the integration of these two parallelisms is an effective approach for achieving more scalability and faster training.

Super-Linear Speedup: The LSS Transformer achieves super-linear scaling in Tables 3 and 4 due to two reasons. First, longer sequences result in increased work for each GPU due to the self-attention computation increase, leading to higher GPU utilization rates for longer sequences and super-linear scaling. Measured by PyTorch Cuda utilization report, GPU utilization rate for the LSS Transformer increases from 33% at 1 node with a sequence length of 348 to 83% at 6 nodes with a sequence length of 2,088. Second, the LSS Transformer's low communication overhead significantly contributes to its excellent parallel efficiency. Fig. 4(a) shows the scaling efficiency with and without the fused communication and gradient averaging techniques, which were both introduced in Sec. 3. At 864 GPUs and sequence length of 50,112, the scaling efficiency with both techniques are 151%. Efficiency with gradient averaging but without fused communication is 147%, whereas efficiency without either technique is dropped to 118%.

Maximal Sequence Length: Fig. 4(b) depicts the maximal sequence length when scaling the number of GPUs while maximizing memory capacity. Each numbered pair in the figure corresponds to the number of GPUs and the maximal sequence length. For example, $(6, 0.59)$ indicates that 6 GPUs can scale up to a maximal sequence length of 0.59×10^4. We can observe that the maximal sequence length follows a square root curve in the graph. Since transformer has quadratic memory complexity with longer sequences, the maximal sequence length increases asymptotically with square root functions as the total memory capacity grows.

Runtime Breakdown. Fig. 4(c) illustrates the runtime breakdown for the weak scaling results presented in Table 3(a), focusing on a single data parallel group. The blue bars represent the percentage of runtime spent on computations, while the orange bars indicate the percentage of runtime for communications. The grey bars denote the GPU waiting times. At 36 GPUs, the communication overhead accounts for 27% of the total runtime. As the number of GPUs scales to 864 (144 nodes), the communication overhead becomes 40% of the runtime.

Scaling Sub-Linearly Increased Sequence Lengths. One way to limit memory and computations increase is to increase the sequence lengths at a sub-linear rate. Table 5 repeats the same scaling experiment as Table 3, but with the sequence lengths increasing proportionally to the square root of the number of GPUs. As a result, both memory and self-attention computations from rows 5 and 7 of the table also increase at a rate asymptotic to square root functions. The memory footprint for the LSS-Transformer is only at 1.15 GB per GPU when scaling to 864 GPUs, and the self-attention computations is only 10 times more than that for a single node. The LSS-Transformer's scaling remains highly efficient at 94% with 864 GPUs. In contrast, scaling efficiency for Nvidia's sequence parallelism drops to 31% at 108 GPUs and cannot scale further due to memory constraints.

5.3 Large Model Experiment

Table 6 repeats the same experiment as Table 5, but trains a large 1.5-billion-parameter GPT model that has a 2048 embedding size, 24 attention layers and 16 multi-heads. experiments are run on the high-memory nodes of Summit with 32 GBs of memory per GPU, and no model parallelism is used for this experiment. Since most of the memory capacity is now used for storing model parameters instead of long sequences, we can notice that all runs in this table use much more memory than the small model experiment. LSS Transformer maintains a high scaling efficiency of 92% at 108 GPUs and a smaller memory footprint compared to the baseline parallelism. In contrast, the baseline parallelism cannot scale beyond a single node due to memory constraint an its FLOPs throughput is 2.3 times smaller than that for LSS Transformer at a single node.

6 Conclusion

This paper introduced the Long-Short Sequence Transformer (LSS Transformer), a novel algorithm and a general framework for distributing long sequence in transformer models. It uses a novel distributed self-attention mechanism, along with fused communication and double gradient averaging techniques, to achieve impressive speedups and memory reduction with minimal communication overhead.

In conclusion, the LSS Transformer is a significant step forward for addressing transformer's long sequence problem. We believe that our approach provides an important contribution to the research field and enables ultra-long sequence training, especially to applications that benefit from long-range token dependencies, such as DNA sequence analysis, long document summary, and imaging applications.

References

Al-Rfou, R., Choe, D., Constant, N., Guo, M., and Jones, L. Character-level language modeling with deeper self-attention. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01): 3159–3166, July 2019. doi: 10.1609/aaai.v33i01.33013159. URL https://ojs.aaai.org/index.php/AAAI/article/view/4182.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The

Table 6. Performance comparison between the two algorithms on a large 1.5 billion model, using only 1 data parallel group, and without any model parallelism. Sequence lengths increase with a growth rate proportional to the square root of the number of GPUs.

(a) LSS Transformer, data parallel group = 1

Nodes	1	6	18
GPUs	6	36	108
Sequence Groups	6	36	108
Sequence Length	366	900	1512
Per GPU Mem. (GB)	21.67	22.48	23.34
FLOPS (x10^{12} flop/s)	52	518	2010
Self-Attn Comp Incr.	1	2	4
Parallel Efficiency	100%	94%	92%

(b) Baseline, data parallel group = 1

Nodes	1	6
GPUs	6	36
Sequence Groups	6	36
Sequence Length	366	900
Per GPU Mem. (GB)	25.28	OOM
FLOPS (x10^{12} flop/s)	23	OOM
Self-Attn Comp Incr.	1	OOM
Parallel Efficiency	100%	OOM

long-document transformer, 2020.

Caldarini, G., Jaf, S., and McGarry, K. A literature survey of recent advances in chatbots. *Information*, 13(1):41, 2022.

Chen, B., Dao, T., Winsor, E., Song, Z., Rudra, A., and Ré, C. Scatterbrain: Unifying sparse and low-rank attention. *Advances in Neural Information Processing Systems*, 34: 17413–17426, 2021a.

Chen, C.-F. R., Fan, Q., and Panda, R. Crossvit: Cross-attention multi-scale vision transformer for image classification. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 357–366, New York, NY, USA, 2021b. IEEE.

Chen, R. J., Chen, C., Li, Y., Chen, T. Y., Trister, A. D., Krishnan, R. G., and Mahmood, F. Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16123–16134, New York, NY, USA, 2022. IEEE. doi: 10.1109/CVPR52688.2022.01567.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers, 2019. URL https://arxiv.org/abs/1904.10509.

Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., and Weller, A. Rethinking attention with performers. In *The International Conference on Learning Representations (ICLR)*, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.48550/ARXIV.2009.14794. URL https://arxiv.org/abs/2009.14794.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS: Proceedings of the 35th Neural Information Processing Systems Conference*, New York, NY, USA, 2022. Association for Computing Machinery. doi: 10.48550/ARXIV.2205.14135. URL https://arxiv.org/abs/2205.14135.

Dong, L., Xu, S., and Xu, B. Speech-transformer: A no-recurrence sequence-to-sequence model for speech recognition. In *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 5884–5888, New York, NY, USA, 2018. IEEE.

Hutter, M., Mahoney, M., and Bowery, J. 500'000 €prize for compressing human knowledge. http://prize.hutter1.net/, 2006. Accessed: 2023-03-05.

Jacobs, S. A., Tanaka, M., Zhang, C., Zhang, M., Song, S. L., Rajbhandari, S., and He, Y. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models, 2023.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, New York, NY, USA, 2020. Association for Computing Machinery.

Kitaev, N., Kaiser, L., and Levskaya, A. Reformer: The efficient transformer. In *The International Conference on Learning Representations (ICLR)*, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.48550/ARXIV.2001.04451. URL https://arxiv.org/abs/2001.04451.

Korthikanti, V., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models, 2022. URL https://arxiv.org/abs/2205.05198.

Li, D., Shao, R., Xie, A., Xing, E. P., Gonzalez, J. E., Stoica, I., Ma, X., and Zhang, H. Lightseq: Sequence level parallelism for distributed training of long context transformers, 2023.

Li, S., Xue, F., Baranwal, C., Li, Y., and You, Y. Sequence parallelism: Long sequence training from system perspective, 2021. URL https://arxiv.org/abs/2105.13120.

Mahoney, M. Enwik8 test data. https://mattmahoney.net/dc/textdata.html, 2006. Accessed: 2023-03-05.

Rabe, M. N. and Staats, C. Self-attention does not need $o(n^2)$ memory, 2022.

Roy, A., Saffar, M., Vaswani, A., and Grangier, D. Efficient Content-Based Sparse Attention with Routing Transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 02 2021. ISSN 2307-387X. doi: 10.1162/tacl_a_00353. URL https://doi.org/10.1162/tacl_a_00353.

Shi, H., Gao, J., Ren, X., Xu, H., Liang, X., Li, Z., and Kwok, J. T. Sparsebert: Rethinking the importance analysis in self-attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9547–9557, New York, NY, USA, 2021. PMLR. URL http://proceedings.mlr.press/v139/shi21a.html.

Si, Y. and Roberts, K. Three-level hierarchical transformer networks for long-sequence and multiple clinical documents classification, 2021. URL https://arxiv.org/abs/2104.08444.

Strudel, R., Garcia, R., Laptev, I., and Schmid, C. Segmenter: Transformer for semantic segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 7262–7272, New York, NY, USA, 2021. IEEE.

Sukhbaatar, S., Grave, E., Bojanowski, P., and Joulin, A. Adaptive attention span in transformers. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 331–335, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1032. URL https://aclanthology.org/P19-1032.

Valanarasu, J. M. J., Oza, P., Hacihaliloglu, I., and Patel, V. M. Medical transformer: Gated axial-attention for medical image segmentation. In *Medical Image Computing and Computer Assisted Intervention–MICCAI 2021: 24th International Conference, Strasbourg, France, September 27–October 1, 2021, Proceedings, Part I 24*, pp. 36–46, New York, NY, USA, 2021. Springer.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pp. 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., and Chao, L. S. Learning deep transformer models for machine translation. In *57th Annual Meeting of the Association for Computational Linguistics*, pp. 1810–1822, New York, NY, USA, 2019. Association for Computing Machinery. doi: 10.48550/ARXIV.1906.01787. URL https://arxiv.org/abs/1906.01787.

with Codes, P. Language modelling on enwik8. https://paperswithcode.com/sota/language-modelling-on-enwiki8, 2022. Accessed: 2023-03-05.

Ying, C., Ke, G., He, D., and Liu, T.-Y. Lazyformer: Self attention with lazy update, 2021.

Yu, J., Li, J., Yu, Z., and Huang, Q. Multimodal transformer with multi-view visual representation for image captioning. *IEEE Transactions on Circuits and Systems for Video Technology*, 30(12):4467–4480, 2019.

Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. Megabyte: Predicting million-byte sequences with multiscale transformers, 2023.

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33:17283–17297, 2020.